Object detection using DOTA dataset and training using YOLOV8

ANN NIBANA S L

1 Dataset Preparation and Conversion

- Downloaded **DOTA-v1.0** dataset and extracted image and labelTxt.
- Selected **40 images** with aeroplane using a Python script.
- Used **dotadevkit** to convert the dataset to **COCO format** → generated DOTA_1.0.json.
- Split the dataset:
  - 30 images to images/train
  - 10 images to images/validation
- Created corresponding **COCO-style JSON** files: instances_train.json, instances_val.json

2. Folder Structure and Configuration

Organized dataset as:

mini_train/

├── images/

| ├── train/

| └── val/

├── labels/

| ├── train/

| └── val/

├── annotations/

| ├── instances_train.json

| └── instances_val.json

Created a data.yaml file

3. YOLOv8 Setup, & Training

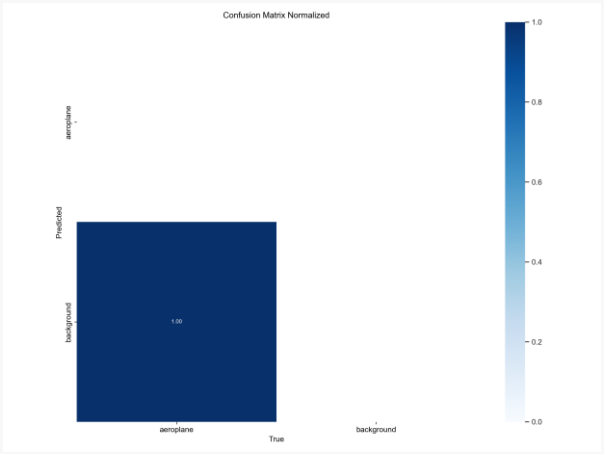
- Installed **Python 3.9** and YOLOv8 using pip install ultralytics.
- Resolved:
  - CUDA compatibility (GPU verified)
- Fixed class ID issues in label .txt files (converted all class IDs to 0).
- Resized all training/validation images to **640×640**.

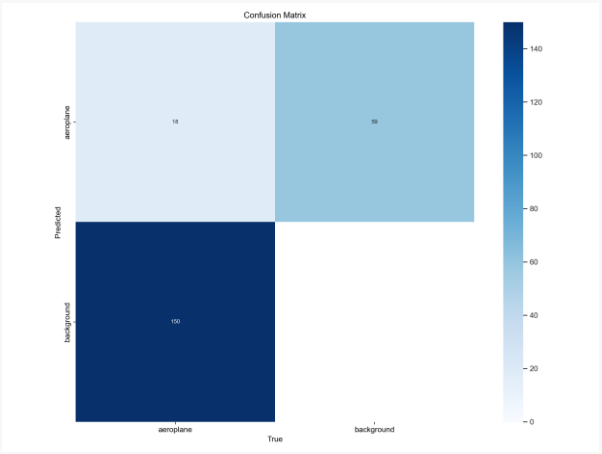
4. Training Models

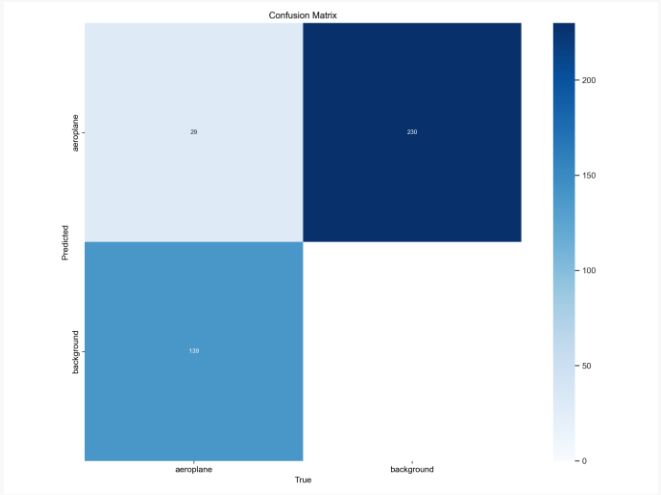
Epoch10(none)

Epoch30(11)

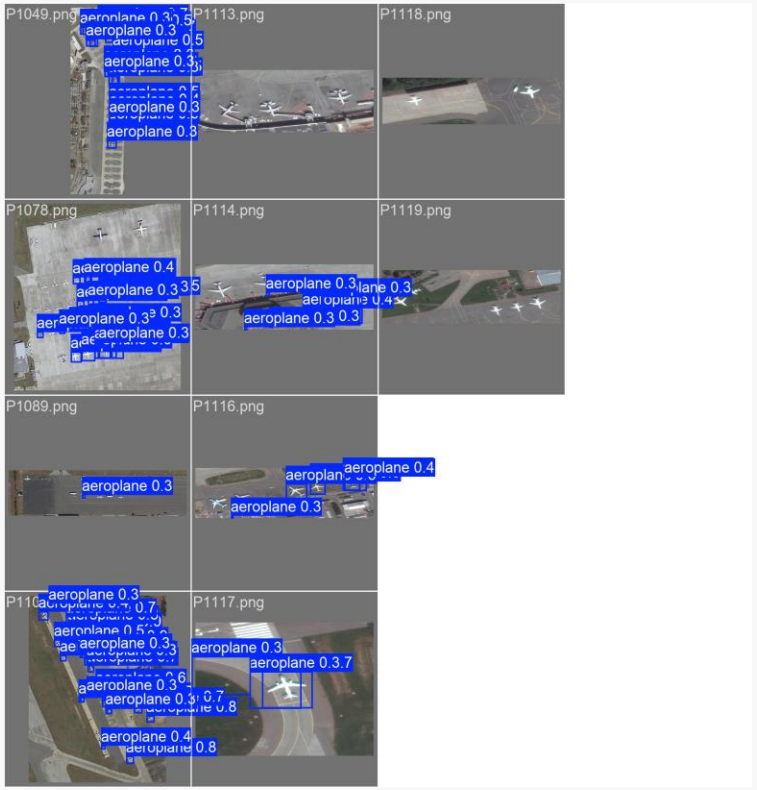
Epoch50(29)

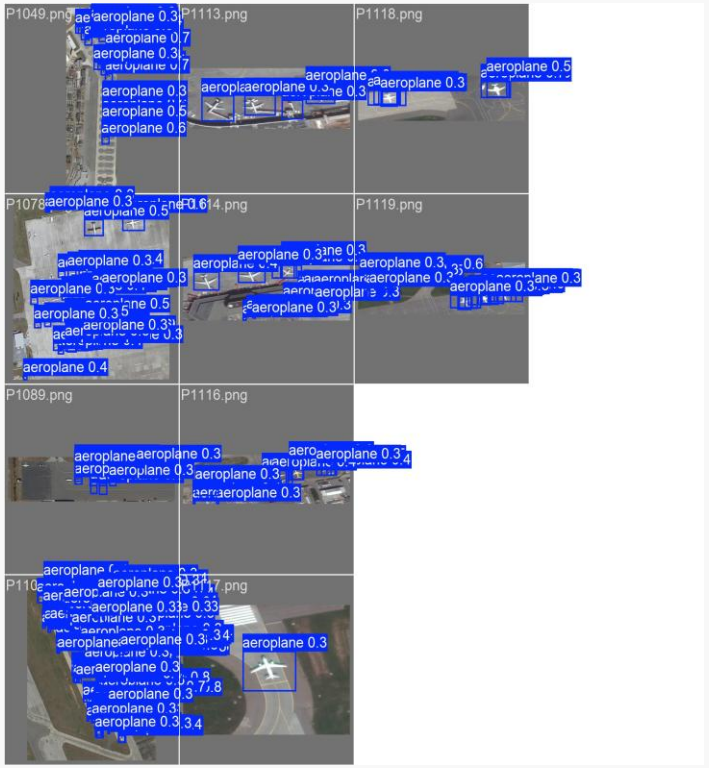






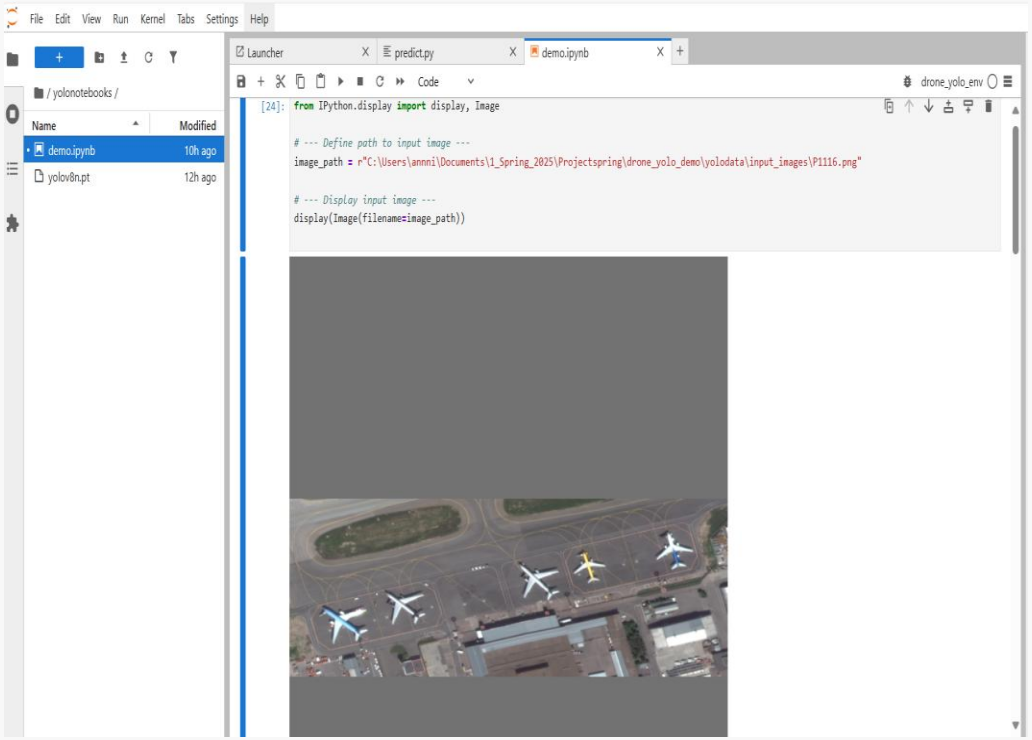

P1049.png
P1113.png
P1118.png
P1078.png
P1114.png
P1119.png
P1089.png
P1116.png
P1109.png
P1117.png


P1049.png
P1113.png
P1118.png
P1078.png
P1114.png
P1119.png
P1089.png
P1116.png
P1117.png


P1049.png
P1113.png
P1118.png
P1078.png
P1114.png
P1119.png
P1089.png
P1116.png
P1117.png

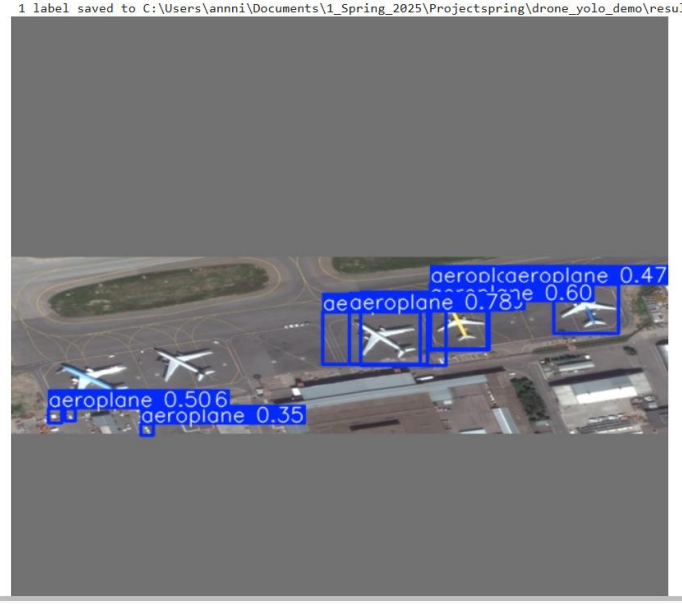
Setting Up a Jupyter environment

- Conda environment named drone_yolo_env
- Necessary packages ultralytics, jupyterlab, and ipython installed
- JupyterLab launched
- A notebook named demo.ipynb used to load and display input drone images using the IPython.display.Image function.
- Using the Epoch30 YoloV8s(best.pt)file performed object detection on a specified input image with a confidence threshold of 0.3.
- Saved to results folder (predict_output2). If the output folder already exists, it is removed to ensure a clean run.
- Displays it using Jupyter's IPython.display.Image function.




```
[30]:
import os
import shutil
from ultralytics import YOLO
from IPython.display import display, Image

# --- Define paths ---
model_path = r"C:\Users\annni\Documents\1_Spring_2025\Projectspring\drone_yolo_demo\weights\best.pt"
output_dir = r"C:\Users\annni\Documents\1_Spring_2025\Projectspring\drone_yolo_demo\results"
predict_name = "predict_output2"
predict_folder = os.path.join(output_dir, predict_name)

# --- Remove old output folder if it exists ---
if os.path.exists(predict_folder):
    shutil.rmtree(predict_folder)

# --- Load YOLOv8 model ---
model = YOLO(model_path)

# --- Run prediction ---
results = model.predict(
    source=image_path,
    save=True,
    save_txt=True,
    project=output_dir,
    name=predict_name,
    imgsz=640,
    conf=0.6
)

# --- Display prediction image ---
predicted_path = os.path.join(results[0].save_dir, os.path.splitext(os.path.basename(image_path))[0] + ".jpg")
display(Image(filename=str(predicted_path)))
```

```
Speed: 9.8ms preprocess, 1196.9ms inference, 4.1ms postprocess per image at shape (1, 3, 640, 640)
Results saved to C:\Users\annni\Documents\1_Spring_2025\Projectspring\drone_yolo_demo\results\predict_output2
1 label saved to C:\Users\annni\Documents\1_Spring_2025\Projectspring\drone_yolo_demo\results\predict_output2\labels
```



Improvements

- Add more imageries to train
- Here worked with YoloV8s try out with Yolov8m
- More Epochs
- Work around with confidence threshold
- Try finding the coordinate locations and overlaying it on a map